SOP: Building, testing and running ragcap on an Apple M5 Pro

Written for: an engineer who builds, tests and operates ragcap on one Apple Silicon Mac. It follows the work in order: pre-work, preparation, implementation, testing, troubleshooting. Every command was run, and every number was measured, on the machine in section 2 between 2026-10-22 and 2026-10-23. ragcap reproduces the design in Kumar's RAG Capstone PDF (LangGraph router, document RAG with grade and rewrite, SQL agent, general agent, Postgres + pgvector, FastAPI, a UI) on local models, and adds what that design did not measure: frozen test sets, repeated runs, and a version-aware retrieval filter with an OFF/ON proof.

1. Result in one table

Measure (llama3.2:3b, lean profile unless stated)	Result on the M5 Pro	Section
Rebuild from zero, all fingerprints	RECREATE OK in about 20 s	6.2
Clean start from a stopped machine	CLEAN START OK	6.1
Router accuracy, tuned set (27) / unseen set (25)	100% / 100%	6.5
Answers correct, tuned / unseen	92.6% / 80.0% (identical over 3 runs each)	6.5
Answers correct, unseen, recommended profile (+ qwen3:8b second-opinion grader)	96.0%	6.5
Superseded document versions retrieved, filter OFF / ON	99/192 (51.6%) / 0/192 (0.0%); control cases 20/20 both	6.3
Scenario catalogue (18 questions + 1 browser test)	12 pass, 1 observed, 5 documented weaknesses, 0 unexpected	6.4
P&ID drawings: tags read, answers	38/39 tags, 4/12 answers	6.7
Time per question: document / database / general	1.1 s / 0.9 s / 0.3 s	6.5
Memory while answering (lean / recommended peak)	~2.7 GB / ~8.2 GB of 24 GB	6.10

2. Environment

Item	Value
Machine	Apple M5 Pro, 15 cores, 24 GB memory, macOS 26.6; no Homebrew, no Docker, no admin password
Project folder	kumar_rag-project/build (the git repository ragcap); documents in build/docs, mirrored to kumar_rag-project/docs
Python	3.12.15 (arm64) from uv; project venv .venv: langgraph 1.2.16, openai 3.37.0, psycopg 3.3.6, pgvector 0.5.1, fastapi, gradio, pandas
Database	PostgreSQL 16.2 + pgvector 0.6.2 from the pip wheel pgserver 0.1.4, run natively by pgctl.sh; data in .pgdata/; 127.0.0.1:5433, TCP only
Models (Ollama 0.40.9, standalone binary in ~/.local/opt/ollama)	llama3.2:3b (all LLM steps), nomic-embed-text (768-dimension embeddings), qwen3:8b (optional second-opinion grader), qwen2.5vl:3b (P&ID drawings)
Ports	11434 Ollama, 5433 Postgres, 8000 (or 8765) web page and API, 7860 Gradio; all bound to localhost
Secrets	.env (mode 600, gitignored): database password; optional LangSmith key

3. Pre-work: decide and check before starting

3.1 Requirements

Requirement	Why	Check
Apple Silicon Mac, 24 GB memory recommended (16 GB works with the lean profile)	models run on the Metal GPU; the recommended profile peaks at ~8.2 GB	About This Mac
About 12 GB free disk	models 10.7 GB (all recommended), Python packages, database	df -h ~
Internet for the downloads only	tools and models are downloaded once; running needs no network and no API key	-
No admin rights needed	every tool installs into the home folder; Postgres comes from a pip wheel	-

3.2 What gets built

Component	What it does
Ingestion (ragcap/ingest.py)	Markdown, PDF (Docling when installed), Word, PowerPoint, HTML to text; paragraphs packed into ~180-word chunks; front-matter (doc, version, effective_date, superseded) becomes metadata and is removed before embedding; CSV/Excel become SQL tables; PNG/JPG drawings are described by a vision model
Router	classifies a question as documents, database or general and rewrites it as a standalone question using the chat history
Document path	retrieve top 6 chunks from pgvector, excluding superseded versions; grade each passage; one rewrite if none is kept; generate a cited answer or 'not in the knowledge base'
Database path	SQL agent writes one SELECT from the table's columns, sample rows and allowed values; it runs read-only with a 5 s timeout and one retry; a second step words the answer from the rows
General path	greetings and questions about the assistant; forbidden from stating facts
Interfaces	FastAPI (/health, /upload, /chat streaming one event per step, /eval), a static web page on /chat, a Gradio chat
Evaluation	frozen golden sets, eval.py, retrieval ablation b1_eval.py, data checks b1_check.py, scenarios.py, pid_eval.py

3.3 Design decisions already built into the code

Decision	Why it is there
Router prompt lists the uploaded tables and their columns; 'general' accepted only for greetings, thanks and questions about the assistant	a single table pulls rule questions ('disk threshold') toward SQL; the guard stops facts from being answered without sources
Standalone question copies the user's words and only resolves pronouns	added words changed what retrieval found
Grader asks one yes/no question per passage	a 3B model cannot reliably pick indices out of six passages in one call
SQL rules: WHERE only for conditions the question names; never GROUP BY the id; LIMIT 1 only for a single extreme; copy numbers exactly	each rule fixes an observed wrong query or a garbled number
JSON from the model is parsed leniently and never raises; model calls retry 3 times on connection errors	one truncated reply must not end a 27-question run
Version filter: WHERE d.superseded = false before the vector ORDER BY, on by default (VERSION_AWARE=1)	an old version is 0.96-0.99 similar to the current one and is otherwise retrieved half the time (section 6.3)

3.4 How results are kept honest

(1) The 27-question golden set was committed before any pipeline code existed, and the 25-question unseen set before its first run; git order is the evidence. (2) Golden questions are never pasted into prompts. (3) A scorer fix goes into golden/amendments.jsonl in its own commit (one entry: doc-008), never into the frozen file, and never after seeing a result it would change. (4) Every change is committed before it is measured, on the full set, and judged by the failing case ids. (5) Scores are reported over 3 repeated runs. (6) recreate.sh compares sha1 fingerprints of the frozen sets, the corpus and the drawings, and the ablation headline, and exits 1 on any difference.

4. Preparation: install the tools (once per Mac)

Full detail, download locations and expected output: docs/INSTALL_AND_USE.md (PDF INSTALL_ragcap.pdf). In short:

```
curl -LsSf https://astral.sh/uv/install.sh | sh && source ~/.zshrc     # uv into ~/.local/bin
uv python install 3.12                                                  # python3.12 (arm64)
mkdir -p ~/.local/opt/ollama && cd ~/.local/opt/ollama
curl -fsSL -o ollama-darwin.tgz https://github.com/ollama/ollama/releases/latest/download/ollama-darwin.tgz
tar xzf ollama-darwin.tgz && rm ollama-darwin.tgz && ln -sf ~/.local/opt/ollama/ollama ~/.local/bin/ollama
mkdir -p ~/.ollama && nohup ollama serve > ~/.ollama/serve.log 2>&1 &
ollama pull llama3.2:3b && ollama pull nomic-embed-text                 # required, 2.3 GB
ollama pull qwen3:8b                                                    # recommended grader, 5.2 GB
ollama pull qwen2.5vl:3b                                                # only for drawings, 3.2 GB
```

Verify: python3.12 --version prints 3.12.x; ollama --version prints 0.40.x; ollama list shows the pulled models.

5. Implementation

All commands run in the build folder. ./recreate.sh performs 5.1 to 5.3 and the checks of section 6.2 in one go; doing them by hand shows what each part does.

5.1 Settings and Python environment.

```
PW=$(openssl rand -hex 12)
printf 'PGHOST=localhost\nPGPORT=5433\nPGUSER=ragcap\nPGPASSWORD=%s\nPGDATABASE=ragcap\n' "$PW" > .env; chmod 600 .env
python3.12 -m venv .venv
.venv/bin/pip install -r requirements.txt -r requirements-local-pg.txt
.venv/bin/python -c "import langgraph, psycopg, pgserver, platform; print('ok', platform.machine())"   # ok arm64
```

5.2 Database and schema. pgctl.sh creates the database on first start with the password from .env and runs it natively (it would use a Docker container instead if the docker command existed). The server listens on TCP only: its Unix socket is off because the project path is longer than macOS's 103-byte socket limit, which otherwise stops the server from starting.

```
./pgctl.sh start && ./pgctl.sh status                       # pg_ctl: server is running
./pgctl.sh psql -c "CREATE EXTENSION IF NOT EXISTS vector;"
./pgctl.sh psql -tAc "select extversion from pg_extension where extname='vector'"     # 0.6.2
.venv/bin/python step1_check.py                             # applies schema.sql, stores and finds two embeddings
   embedding dim: 768 / nearest: Kafka consumer lag alert threshold 0.383 / STEP 1 OK
```

schema.sql: documents (name, version, superseded, effective_date, source_path), chunks (text, embedding vector(768), meta jsonb, HNSW cosine index), uploaded_tables (tables the SQL agent may query). pgvector 0.6.2 has no hnsw.iterative_scan, so retrieve() sets hnsw.ef_search = 1000 instead; a filtered search therefore still returns 6 chunks on a larger table.

5.3 Ingest the corpus and the table. The corpus is 8 Kafka runbooks, each in an old (superseded) and a current version, plus one CSV table.

```
.venv/bin/python -m ragcap.ingest data/docs_versioned       # 16 documents, 1 chunk each
.venv/bin/python -m ragcap.ingest data/tables               # broker_inventory: 6 rows, 6 columns
./pgctl.sh psql -tAc "select count(*) from documents"       # 16; chunks 16; superseded 8; uploaded_tables 1
```

5.4 The graph (ragcap/graph.py). State: question, history, route, standalone question, chunks, relevant chunks, rewrite count, SQL, rows, answer, and a per-step trace (milliseconds, tokens).

Step	What it does	LLM calls
router	JSON {route, standalone}; sees the table names and columns; guard on 'general'	1

Step	What it does	LLM calls
retrieve	embeds the standalone question; cosine search in pgvector; top 6; WHERE superseded = false when VERSION_AWARE=1	0 (1 embedding)
grade	one yes/no call per passage (3 in parallel); with GRADE_MODE=escalate, if nothing is kept, one qwen3:8b call reads all six	6 (+1)
rewrite	one expanded query, allowed once, then retrieve and grade again	1
generate	answer with [n] citations; 'not in the knowledge base' without an LLM call if nothing is relevant	0-1
sql_agent	one SELECT validated (no ';', no write keywords), READ ONLY transaction, 5 s statement timeout, one retry with the error	1-2
generate_sql	plain-language answer from the rows, numbers copied exactly	1
general	short reply, no facts	1

```
.venv/bin/python try.py "Thanks!" "How many brokers are in rack-b?" "At what disk utilisation should a broker page?"
   route=general  ... You're welcome!     route=database ... There are 2 brokers in rack-b.
   route=documents ... exceeds 75%.        (each 0.3-1.5 s)
```

5.5 Interfaces.

```
# web page at http://127.0.0.1:8000/ ; API: GET /health, POST /upload, POST /chat (streaming)
.venv/bin/uvicorn api:app --port 8000
.venv/bin/python ui.py                     # Gradio, http://127.0.0.1:7860
```

5.6 P&ID drawings. pid/make_pid.py draws three synthetic P&IDs from one template and writes their ground truth (39 tags, every valve and instrument relation) and 12 frozen questions. Ingesting a PNG asks the vision model for one fact per sentence (title block, equipment, valves and what they sit between, instruments and what they measure); that text is embedded like any document and the image path is kept in the chunk's metadata.

```
.venv/bin/python pid/make_pid.py                              # deterministic drawings + ground truth + questions
.venv/bin/python -m ragcap.ingest pid/drawings                # ~25 s with qwen2.5vl:3b
```

5.7 Docling (optional). Layout-aware parsing for PDF, Word, PowerPoint and HTML, kept in its own environment so the main one stays small. ingest.py uses Docling if it is importable, otherwise .venv-docling as a subprocess, otherwise pypdf text for PDFs. Measured: a 7-page PDF converted in ~47 s with its tables kept as Markdown tables (torch 2.14.1 on the Metal backend).

```
uv venv --python 3.12 .venv-docling && uv pip install --python .venv-docling/bin/python docling
```

5.8 Operations scripts.

Script	Use
pgctl.sh	the database: start, stop, status, ready, psql, dump, restore, reset
services.sh	start, stop [--all], status, check (the clean-start check of 6.1)
recreate.sh	rebuild from zero and verify every fingerprint (6.2); resets the database to the 16-document baseline
scenarios.py	the scenario catalogue with expected answers (6.4)
run_series.sh, summarize_series.py	repeated evaluation runs and their mean, range and failing ids (6.5)

5.9 Choose the model profile. Lean is the default. The recommended profile adds qwen3:8b as a second-opinion grader (sections 6.5 and 6.10); turn it on with three lines in .env and restart the web page or Gradio:

```
GRADE_MODEL=qwen3:8b
GRADE_MODE=escalate
GRADE_MAX_TOKENS=600
```

6. Testing

6.1 Clean start. Proves the whole path works after a reboot or sleep. Tested from a fully stopped state (results/clean_start_20261023.log):

```
$ ./services.sh stop --all      postgres: stopped / ollama: stopped
$ ./services.sh check           FAIL postgres not ready; FAIL ollama not running; CLEAN START FAILED (2)  [exit 1]
$ ./services.sh start           ollama: started (Ollama 0.40.9); postgres: running
$ ./services.sh check           PASS python venv 3.12.15 arm64 | PASS .env mode 600 | PASS postgres ready
                                PASS pgvector 0.6.2 | PASS data 16 docs, 16 chunks, 8 superseded, 1 tables
                                PASS ollama, llama3.2:3b, nomic-embed-text
                                PASS smoke general 302 ms, database 575 ms, documents 748 ms
                                PASS api /health | PASS web page / HTTP 200 | CLEAN START OK  [exit 0]
```

6.2 Rebuild and fingerprints. ./recreate.sh ends RECREATE OK. Checked: database counts 16 docs, 16 chunks, 8 superseded, 1 table; golden.jsonl sha1 68ec09bd..., golden_heldout.jsonl 02ddcfd6..., amendments.jsonl 36a0e6e0..., corpus c2b025608d03; P&ID questions, ground truth and the three drawings byte for byte; data checks PASS with all three injected faults reported (old version marked current, version stored as text, banner in embedded text); pair similarity old vs current 0.963-0.990, old version ranked first for 1 of 8 plain questions.

6.3 Version filter, retrieval level (b1_eval.py: 32 frozen document cases, 12 version traps and 20 controls, no LLM, deterministic):

Metric	Filter OFF	Filter ON
Superseded chunks retrieved	99/192 (51.6%)	0/192 (0.0%)
Trap cases passing (facts present, no stale value from an old chunk)	2/12	12/12
Trap cases with the stale value arriving from an old chunk	10/12	0/12
Stale string anywhere in context (the current runbook quotes its former value)	10/12	6/12
Control cases passing (must stay flat)	20/20	20/20
Median retrieve time, interleaved, 96 calls each	12-13 ms	12-13 ms (never more than 1 ms apart)

The latency varies by a millisecond from run to run, and every recreate.sh run rewrites results/ablation_summary.md, so this SOP quotes the range rather than one run.

6.4 Scenario catalogue (.venv/bin/python scenarios.py --with-drawings; results/scenarios_20261023_113645.md). KNOWN FAIL marks a documented weakness, listed in section 7.

Id	Question (settings)	Result	Answer / what it shows
G1	Thanks!	PASS 0.3 s	general agent, no facts
G2	largest message size our brokers accept?	PASS 2.4 s	nothing relevant, one rewrite, 'not in the knowledge base'
D1	consumer lag to raise an alert?	PASS 1.6 s	5,000 messages, cited (not the old 10,000)
D2	TLS port of BROKER-EAST-02?	PASS 1.5 s	9093 (not 9092)
D3	replication factor and default retention?	KNOWN FAIL	multi-hop: answers 3, misses 72 hours
S1	brokers above 85% disk usage?	PASS 1.1 s	brokers 3 and 5 (87.2%, 91.8%)
S2	broker with the most topics?	KNOWN FAIL	adds an unasked WHERE status='healthy'; says broker 2, 152
S3	total disk capacity in GB?	PASS 0.7 s	16,000 GB
S4	Delete all rows from the broker_inventory table.	PASS 0.6 s	model wrote DELETE; validator refused; table unchanged
V1-OFF	disk utilisation to page (VERSION_AWARE=0)	observed	75%: the old runbook is retrieved but this corpus's current runbook names its former value, so the model still answers right
V1-ON	same, filter on	PASS 1.0 s	75%

Id	Question (settings)	Result	Answer / what it shows
V2	who owns SEV-1 escalation?	PASS 1.5 s	Streaming Platform team (not Platform Reliability)
R1	highest disk usage and which broker?	PASS 1.0 s	91.8%, broker 5 (database route)
R2	minutes lag must stay high?	KNOWN FAIL	lean grader rejects the passage: 'not in the knowledge base'
R2+	same, recommended profile	PASS 6.9 s	5 minutes (qwen3:8b keeps the passage)
U1	which cloud provider hosts Kafka?	PASS 2.5 s	'not in the knowledge base' (refused, not guessed)
M1	valve isolating pump P-101 suction? (drawings)	KNOWN FAIL	'not in the knowledge base' although the drawing is retrieved
M2	controller and valve for flow into T-102?	KNOWN FAIL	'not in the knowledge base'
W1	browser: brokers above 85% on the web page	PASS	pills router 271 ms, sql_agent 399 ms, generate_sql 362 ms; brokers 3 and 5

6.5 Answer-level evaluation, repeated runs (run_series.sh, 3 runs on each set; every configuration gave identical results in all three runs):

```
./run_series.sh m5 llama3.2:3b                                          # lean profile
GRADE_MODEL=qwen3:8b GRADE_MODE=escalate GRADE_MAX_TOKENS=600 N_HELD=3 N_ORIG=3 \
  ./run_series.sh qwen3esc llama3.2:3b                                  # recommended profile
N_HELD=3 N_ORIG=3 ./run_series.sh llama8b llama3.1:8b                   # bigger answering model, all steps
.venv/bin/python summarize_series.py <label>                            # -> results/<label>_summary.md
```

Measure	Lean (llama3.2:3b)	Recommended (+ qwen3:8b grader)	llama3.1:8b, all steps
Unseen set (25): router / answers	100% / 80.0%	100% / 96.0%	100% / 92.0%
Unseen set: failing ids	h-doc-01, -05, -13, -14, -15	h-doc-15	h-doc-14, h-una-02 (a refusal in other words)
Unseen set: unanswerable refused	2/2	2/2	1/2 as scored
Tuned set (27): answers	92.6% (doc-015, db-004)	92.6% (doc-015, db-004)	96.3% (db-004)
Tuned set: multi-hop	1/2	1/2	2/2
Retrieval hit rate	100%	100%	100%
Time: document / database / general	1.1 / 0.9 / 0.3 s	3.0 / 0.9 / 0.3 s	2.5 / 1.5 / 0.7 s
Memory while answering	~2.7 GB	~2.7 GB, peak ~8.2 GB	~5.5 GB

Reading it: four of the five unseen failures in the lean profile are the 3B grader rejecting a correctly retrieved passage that is phrased differently from the question; the qwen3:8b second opinion fixes all four without making the system guess (both unanswerable questions are still refused). llama3.1:8b fixes multi-hop on the tuned set but is lower on the unseen set and always resident. h-una-02 is scored as a failure because the 8B model refused in other words ('not explicitly mentioned in the provided passages'); the frozen scorer is not changed after seeing a result.

6.6 Version filter, answer level (27-question set, LLM in the loop): filter OFF 88.9% (24/27), filter ON 92.6% (25/27). The difference is multi-hop, 0/2 to 1/2: with old versions excluded, the six retrieved chunks hold more distinct documents. Version questions are 8/8 both ways because each current runbook in this corpus names its former value; the retrieval-level proof (6.3) is the one that matters for a corpus without such sentences.

6.7 P&ID evaluation (pid_eval.py: tags against ground truth, 12 questions through the full graph):

Configuration	Tags (of 39)	Answers (of 12)	Seconds per drawing	Memory
qwen2.5vl:3b, lean (default)	38	4	5-12	5.0 GB while ingesting
qwen2.5vl:7b	39	6	7-11	7.7 GB while ingesting
qwen2.5vl:7b + qwen3:8b grader	39	6	-	-
qwen2.5vl:7b + llama3.1:8b answering	39	5	-	-

The right drawing is retrieved for 12/12 questions in every configuration, and the 7b descriptions are complete ('HV-101: Hand (isolation) valve between T-101 and P-101'). A stage probe showed the grader keeping the right drawing while the answer step still replied 'not in the knowledge base': the questions need reasoning over connections (which valve is on the suction side). Neither a bigger grader nor a bigger answering model fixes that; storing each drawing's connections as table rows the SQL agent can query is the next step (section 9).

6.8 Interfaces, upload, tracing. In headless Chromium: the web page showed the three step pills and brokers 3 and 5 for 'Which brokers are above 85% disk usage?' (1035 ms); Gradio answered 75% with the trace 'router 265ms > retrieve 62ms > grade 437ms > generate 154ms' (screenshots in docs/img). POST /upload of a Markdown file returned it ingested and /health rose by one document. LangSmith opt-in: with LANGSMITH_TRACING=true, a bogus key and an unreachable endpoint, the question was still answered (exit 0) and the tracer only logged a connection warning; off by default.

6.9 Backup and restore. ./pgctl.sh dump > file then ./pgctl.sh restore < file kept all 16 chunks.

6.10 Resources, measured (resident size of Ollama's runner process with one model loaded; Ollama's own 'ollama ps' under-reports the vision models):

Model	Role	Disk	Memory	Keep?
llama3.2:3b	all LLM steps	2.0 GB	2.4 GB	required
nomic-embed-text	embeddings	0.3 GB	0.3 GB	required (changing it means re-ingesting)
qwen3:8b	second-opinion grader	5.2 GB	5.5 GB	recommended; loaded only for hard questions
qwen2.5vl:3b	drawings at ingest	3.2 GB	5.0 GB	only if you ingest drawings
llama3.1:8b, qwen2.5vl:7b	tested in 6.5 and 6.7	4.9 / 6.0 GB	5.2 / 7.7 GB	removed (ollama rm); not worth the memory

Ollama unloads a model after 5 minutes idle, so the recommended profile uses ~2.7 GB most of the time and ~8.2 GB for a few minutes after a hard question.

7. Known weaknesses (all reproducible with scenarios.py)

Weakness	Cases	Cause	Lever
Grader rejects a differently phrased passage	R2, h-doc-01/05/13/14	3B per-passage grader	recommended profile (fixed: 80% to 96% unseen)
Multi-hop: two facts from two documents	D3, doc-015, h-doc-15	grader keeps one of the two passages	split the question and retrieve per part
SQL adds an unasked condition or grouping	S2, db-004	3B model writing SQL	a check that re-runs SQL with no extra WHERE when the result looks implausible
P&ID relations	M1, M2	answer step does not infer connections	connections as structured rows for the SQL agent

8. Troubleshooting: problems met on this machine and their fixes

Symptom	Cause	Fix
A copied .venv does not run (wrong architecture or a missing interpreter path)	virtual environments are tied to the machine and user that made them	mv .venv .venv-old; python3.12 -m venv .venv; pip install -r requirements.txt -r requirements-local-pg.txt
pg_ctl: could not start server; log: Unix-domain socket path is too long	macOS limits socket paths to 103 bytes; the project path is longer	pgctl.sh starts the server with unix_socket_directories='' (TCP only)
docker: command not found in every script	Docker Desktop needs admin rights and is not installed	pgctl.sh native backend; all scripts call pgctl.sh
git: 'You have not agreed to the Xcode license agreements'	the Xcode license is not accepted on this Mac	sudo xcodebuild -license accept (needs an admin); git is only needed to commit or push
uvicorn: address already in use on 8000	another program owns port 8000	lsof -nP -iTCP:8000 -sTCP:LISTEN; use --port 8765
An evaluation stood still for 16 minutes, then finished	the Mac went to idle sleep (pmset -g log: 'Entering Sleep state due to Idle Sleep')	run long commands under caffeinate -i; the series scripts do it themselves
P&ID summary named the wrong vision model	ingest.py did not return the model name; pid_eval.py fell back to a fixed name	fixed: ingest returns vision_model; check what ran with curl -s localhost:11434/api/ps
results/pid_eval_summary.md shows a different run than expected	every pid_eval.py run overwrites it	copy it before an experiment (the default 3b run is kept as pid_eval_summary_m5_3b.md)
'delete all brokers' routed to documents	short imperative without the table name	name the table; the SQL validator then refuses the DELETE (scenario S4)
ollama ps shows 0.8 GB for a vision model	Ollama under-reports vision runners	measure with ps -axo rss,command | grep ollama

More symptoms, and the stage-by-stage probe for a wrong answer, are in the runbook (RUNBOOK_ragcap_operations.pdf, sections 5 and 7).

9. Pending

Deployment, at the end: (a) push the repository and publish the showcase site (git needs the Xcode license accepted); (b) Vercel: import the repository once in the Vercel dashboard (Framework 'Other', output directory site); (c) LangSmith: a real API key in .env. Improvements: (d) P&ID connections as structured rows; (e) multi-hop by splitting a question; (f) an SQL plausibility retry (db-004); (g) real P&ID drawings with their own golden questions; (h) Docling measured on a real PDF set.